

Linux / Git 보고서

로봇(휴머노이드) 수습단원 이주원

1. Linux

a. 리눅스란?



리눅스란 리누스 토르발스가 만든 UNIX기반의 오픈소스 운영체제 중 하나로, 다양한 배포판이 존재한다. 사용 목적과 특징에 따라 크게 데비안 계열, 레드햇 계열, 슬랙웨어 계열 등으로 나뉜다.

b. 리눅스 계열



-최초의 리눅스 배포판: SLS

1992년 5월 피터 맥도날드에 의해 만들어진 소프트랜딩 리눅스 시스템이 리눅스 최초의 배포판이다. 출시 당시에는 가장 인기 있는 리눅스 배포판이었지만 사용자들에 의해 버그가 다소 존재하는 것으로 파악되었고 이것은 곧 다른 리눅스 배포판의 등장을 알리게 되는 계기가 되었다. 패트릭 볼커딩은 SLS에 존재하는 버그를 잡기 시작하는데 이 결과로 만들어진 리눅스 배포판이 슬랙웨어이다.

-슬랙웨어 (Slackware)

현재까지 사용되고 배포되는 가장 오래된 배포판이다. 가장 오래된 만큼 Absolute Linux, Salix os, Unraid 등 많은 OS가 슬랙웨어 계열이다.

-수세 (OpenSUSE)

수세 리눅스는 독일에서 출시된 배포판이다. 오픈 수세는 원래 슬랙웨어 기반으로 시작했지만 현재는 슬랙웨어와 관계가 멀어지고 .rpm을 사용하기에 영미권에서는 독자적인 계열로 취급한다. 상당히 안정적인 리눅스라는 장점이 있지만 저장소가 우분투나 민트에 비해 많지 않다는 단점이 존재한다.

-데비안 (Debian)

데비안 계열 또한 상당히 오래되었다. 패키지 형식은 .deb이며, 패키지 관리자로 apt를 이용한다. 가장 광대하고 많은 리눅스 배포판을 가지고 있고, 그만큼 사용자 수도 많다. 그 중 가장 유명한 것은 우분투이다. 데비안의 장점으로 배포되고 있는 서버 리눅스 들 중에서 안정성이 매우 높고 다양한 패키지 활용이 가능하다.



debian

-우분투 (Ubuntu)

가장 많이 쓰이고 널리 알려진 배포판이다. 앞서 언급했듯 데비안 리눅스를 기반으로 제작되었다. 가장 널리 쓰이는 만큼 이 역시 상당한 양의 파생 배포판이 존재한다. 참고로 북한에서 제작한 자체 OS 인 붉은별 OS가 이것을 바탕으로 만들어졌다.



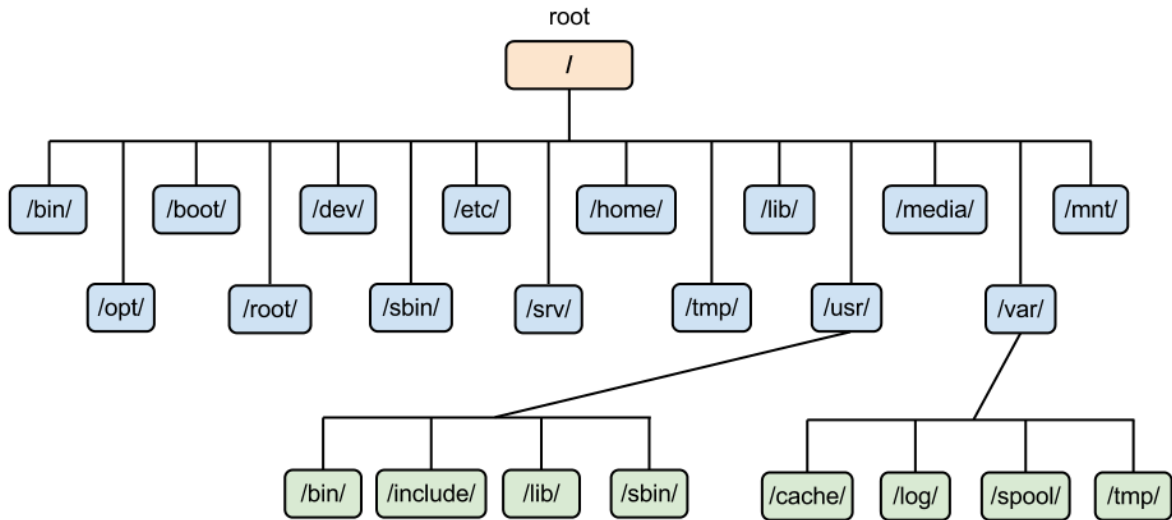
-레드햇 (Red Hat)

패키지 형식은 .rpm이며 패키지 관리자로 yum/dnf를 사용하는 것이 특징이다. 서버나 워크스테이션용으로 사용되는 경우가 대부분이다. 얼마 없는 상용 리눅스 중에서도 굉장히 잘나가는 편인데, 이는 서버 및 워크스테이션 시장이 주 타겟인 배포판이기 때문이다.

-페도라 (Fedora)

레드햇이 9.0버전 이후로 무료버전의 레드햇을 배포하지 않았는데 그 대신 페도라 프로젝트를 후원함으로써 일반인들도 계속 레드햇 리눅스의 연장선에 있는 페도라 리눅스를 사용할 수 있게 했다. 장점으로 는 최신 기술을 체험해 볼 수 있다. 레드햇에서 개발되는 기술이 제일 먼저 탑재되고 개발을 위한 도구들이 기본적으로 제공된다. 단점은 예 려가 많다는 것이다.

c. 리눅스 파일시스템 구조



리눅스는 윈도우와 달리 C드라이브, D드라이브처럼 드라이브 개념이 없고, 최상위에 루트(/) 디렉토리 하나가 있고 그 아래로 모든 파일과 디렉토리가 트리 구조로 연결되어 있다. 이런 디렉토리 구조 규칙을 FHS(Filesystem Hierarchy Standard)라고 부르며, 대부분의 배포판이 이 규칙을 따른다.

-/bin

시스템을 부팅하거나 운영하는 데 필요한 기본 실행 명령어들이 저장되는 디렉토리이다. ls, cp, mv 같은 기본 명령어들의 실행파일이 여기에 위치한다.

-/etc

시스템 설정 파일들이 모여있는 디렉토리이다. 네트워크 설정, 사용자 계정 정보, 각종 서비스의 설정 파일 등이 이곳에 저장되며 관리자가 직접 수정하는 경우가 많다.

-/home

일반 사용자들의 개인 디렉토리가 위치하는 곳이다. 사용자 계정이 생성되면 /home/사용자이름 형태로 개인 폴더가 자동으로 만들어진다.

-/root

루트(관리자) 계정의 홈 디렉토리이다. 일반 사용자의 /home과 분리되어 있으며 최고 관리자 권한을 가진 계정만 접근할 수 있다.

-/var

가변적인(variable) 데이터가 저장되는 디렉토리이다. 로그 파일, 캐시, 메일 등 시스템 운영 중 계속 크기가 변하는 파일들이 여기 저장된다.

-/usr

사용자가 설치한 프로그램과 라이브러리들이 저장되는 디렉토리이다. /bin이 시스템 필수 명령어라면 /usr/bin은 추가로 설치된 프로그램들의 실행파일이 위치하는 곳이라 보면 된다.

-/lib

시스템과 /bin, /sbin의 실행파일들이 필요로 하는 라이브러리 파일들이 저장되는 디렉토리이다.

-/dev

리눅스에서는 하드디스크, USB, 키보드 같은 장치들도 파일처럼 취급하는데, 이런 장치 파일들이 저장되는 디렉토리이다.

-/proc

실제 디스크에 존재하는 디렉토리가 아니라 커널이 실행 중인 프로세스나 시스템 정보를 가상 파일 형태로 보여주는 디렉토리이다. 현재 실행 중인 프로세스 정보 등을 확인할 수 있다.

-/tmp

임시 파일들이 저장되는 디렉토리이다. 재부팅 시 초기화되는 경우가

많다.

-/opt

기본 패키지 관리자를 통하지 않고 별도로 설치하는 추가 소프트웨어 패키지들이 위치하는 디렉토리이다.

d. 기본 명령어

-ls

현재 디렉토리에 있는 파일과 폴더 목록을 보여주는 명령어이다. ls -l 을 사용하면 권한, 소유자, 크기, 수정 날짜 등 상세 정보를 함께 볼 수 있고, ls -a를 사용하면 숨김 파일까지 모두 표시된다.

-cd

디렉토리를 이동할 때 사용하는 명령어이다. cd 디렉토리명으로 해당 디렉토리로 이동하며, cd ..은 상위 디렉토리로, cd ~은 홈 디렉토리로 이동한다.

-pwd

현재 위치한 디렉토리의 절대 경로를 출력해주는 명령어이다. 현재 어느 디렉토리에 있는지 헛갈릴 때 사용한다.

-mkdir

새 디렉토리를 생성하는 명령어이다. mkdir 폴더명 형태로 사용하며, mkdir -p를 사용하면 상위 디렉토리가 없어도 한번에 여러 단계의 디렉토리를 만들 수 있다.

-rm

파일이나 디렉토리를 삭제하는 명령어이다. 디렉토리를 삭제할 때는 rm -r을 사용해야 하며, 강제로 삭제하고 싶을 때는 rm -rf를 사용한다.

복구가 안 되기 때문에 사용할 때 주의가 필요하다.

-cp

파일이나 디렉토리를 복사하는 명령어이다. cp 원본 대상 형태로 사용하며, 디렉토리를 복사할 때는 cp -r을 사용한다.

-mv

파일이나 디렉토리를 이동하거나 이름을 바꿀 때 사용하는 명령어이다. 같은 디렉토리 안에서 사용하면 이름 변경, 다른 디렉토리를 대상으로 하면 이동이 된다.

-cat

파일의 내용을 화면에 출력해주는 명령어이다. 여러 파일을 이어붙여 출력하거나, 짧은 파일을 빠르게 확인할 때 자주 사용한다.

-grep

파일 내용 중 특정 문자열을 검색하는 명령어이다. grep 검색어 파일명 형태로 사용하며, -r 옵션을 추가하면 하위 디렉토리까지 재귀적으로 검색할 수 있다.

-find

특정 조건에 맞는 파일이나 디렉토리를 찾는 명령어이다. find 경로 -name 파일명 형태로 이름을 기준으로 검색할 수 있고, 파일 크기나 수정 시간 등 다양한 조건으로도 검색이 가능하다.

-man

명령어의 사용법과 옵션을 확인할 수 있는 매뉴얼 명령어이다. man 명령어 형태로 사용하며, 리눅스 명령어를 잘 모를 때 가장 먼저 참고하게 되는 명령어이다.

e. 파일 권한과 사용자 관리

리눅스는 다중 사용자 시스템이기 때문에 각 파일과 디렉토리마다 누가 접근할 수 있는지를 세밀하게 관리하는 권한 체계가 존재한다.

-권한의 종류

리눅스의 파일 권한은 읽기(r, read), 쓰기(w, write), 실행(x, execute) 세 가지로 구성된다. ls -l로 파일을 확인하면 -rwxr-xr-- 같은 형태로 표시되는데, 앞의 -는 파일 종류(디렉토리면 d)를 의미하고, 그 뒤로 소유자(owner), 그룹(group), 기타 사용자(other) 순으로 각각 3자리씩 권한이 표시된다.

-chmod

파일이나 디렉토리의 권한을 변경하는 명령어이다. 권한은 숫자로도 표현할 수 있는데 읽기는 4, 쓰기는 2, 실행은 1의 값을 가지며 이를 더해서 표현한다. 예를 들어 chmod 755 파일명을 실행하면 소유자는 읽기·쓰기·실행($7=4+2+1$) 권한을, 그룹과 기타 사용자는 읽기·실행($5=4+1$) 권한을 갖게 된다.

-chown

파일이나 디렉토리의 소유자를 변경하는 명령어이다. chown 사용자명 파일명 형태로 사용하며, 소유자와 그룹을 동시에 바꾸고 싶을 때는 chown 사용자명:그룹명 파일명 형태로 사용한다.

-사용자와 그룹

리눅스에서는 각 사용자가 고유한 계정을 가지며, 여러 사용자를 묶어

그룹으로 관리할 수 있다. /etc/passwd에는 사용자 계정 정보가, /etc/group에는 그룹 정보가 저장되어 있다. useradd, usermod, groupadd 같은 명령어로 사용자와 그룹을 생성·수정할 수 있다.

-sudo

일반 사용자가 관리자(root) 권한이 필요한 명령어를 실행할 때 사용하는 명령어이다. 매번 root 계정으로 로그인하지 않고도 임시로 관리자 권한을 빌려 명령을 실행할 수 있게 해주며, 시스템 설정 변경이나 패키지 설치처럼 일반 사용자 권한으로는 실행할 수 없는 작업에 주로 사용된다.

2. Git



a. Git은...

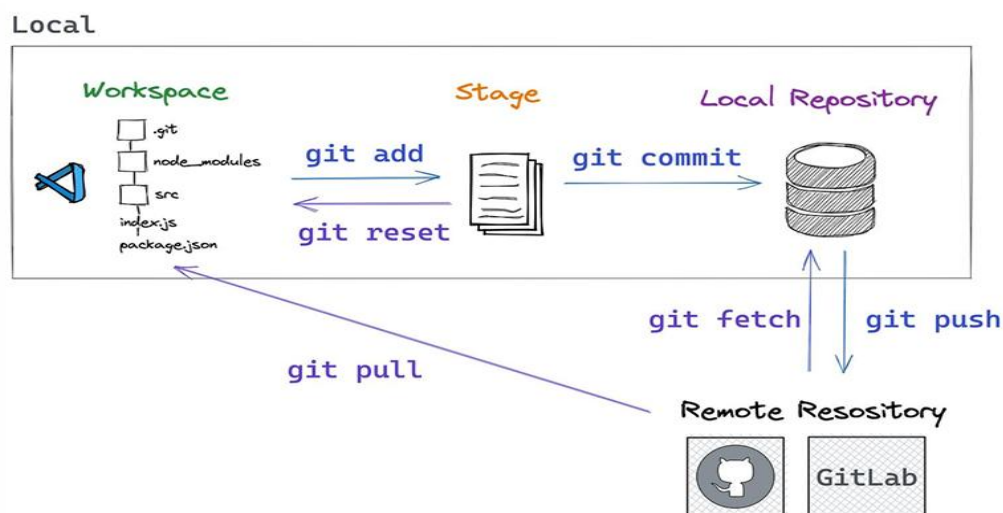
Git 없이 파일을 관리하면 프로그램_v1, 프로그램_v2, 프로그램_최종, 프로그램_최종_진짜 처럼 파일을 계속 복사해서 버전을 남기게 된다. 이런 방식은 시간이 지날수록 어떤 파일이 최신인지 헷갈리고, 누가 무엇을 언제 수정했는지도 알 수 없으며, 여러 사람이 동시에 작업하면 파일이 서로 덮어써지는 문제가 생긴다. Git을 사용하면 파일을 여러 개로 복제하는 대신 .git이라는 숨김 폴더 안에 수정 기록 자체를 저장하기 때문에, 파일 하나를 유지하면서도 "어제 상태로 되돌려줘", "지난주 버전으로 복구해줘" 같은 작업이 가능해진다.

b. Git을 사용했을 때의 장점

모든 수정 이력이 시간 순서대로 기록되기 때문에 언제 어떤 기능이 추가되고 수정되었는지 추적할 수 있다. 또한 코드에 문제가 생겼을 때 이전 정상 상태로 손쉽게 복구할 수 있고 여러 명이 각자 다른 기능(로그인, 회원가입, UI 수정 등)을 동시에 개발한 뒤 나중에 하나로 합칠 수 있어 협업에 최적화되어 있다. 저장소를 GitHub나 GitLab 같은 원격 서버에도 함께 올려두기 때문에 로컬 PC가 고장 나더라도 코드가 유실되지 않는다.

c. Git의 기본 구조

Git은 기본적으로 개발자 PC -> 서버 -> 다른 개발자 PC 형태의 흐름으로 동작한다. 한 개발자가 코드를 수정하고 서버에 업로드하면, 다른 개발자는 그 서버에서 최신 코드를 내려받아 작업을 이어간다. 이런 구조 덕분에 여러 사람이 같은 프로젝트를 동시에, 그리고 안전하게 협업할 수 있다.



d. Repository (저장소)

프로젝트의 모든 파일과 그 변경 이력이 저장되는 공간이다. 내 컴퓨터에 있는 로컬 저장소와, GitHub 같은 서버에 있는 원격 저장소로 나뉜다. `git init` 명령어를 실행하면 현재 폴더가 Git 저장소로 초기화되면서 숨김 폴더인 `.git`이 생성되고, 이 폴더 안에 모든 수정 기록이 저장된다.

e. Commit (커밋)

파일의 변경 사항을 하나의 스냅샷으로 저장하는 것을 의미한다. 코드를 수정한 뒤 `git commit`을 실행하면 "언제, 무엇을, 왜 수정했는지"가 기록으로 남는데, 이때 커밋 메시지를 함께 작성해서 어떤 변경인지 알아볼 수 있게 한다. 커밋은 계속 쌓이면서 프로젝트의 역사(히스토리)를 이루기 때문에, 문제가 생겼을 때 특정 시점의 커밋으로 되돌아갈 수 있다.

f. Branch (브랜치)

같은 저장소 안에서 서로 다른 작업을 독립적으로 진행할 수 있게 해주는 갈래이다. 기본적으로 `main`(또는 `master`)이라는 브랜치가 있고, 새로운 기능을 개발할 때는 별도의 브랜치를 만들어 그 안에서 작업한 뒤, 완성되면 원래 브랜치에 합친다. 이렇게 하면 여러 사람이 각자 다른 기능을 동시에 개발해도 서로의 작업을 방해하지 않는다.

g. Merge (병합)

서로 다른 브랜치에서 작업한 내용을 하나로 합치는 작업이다. 예를 들어 로그인 기능을 개발한 브랜치를 `main` 브랜치에 병합하면, 그 변

경 사항이 main 브랜치에도 반영된다. 만약 같은 파일의 같은 부분을 서로 다르게 수정했다면 충돌(conflict)이 발생하는데, 이 경우 어느 쪽 내용을 남길지 직접 선택해서 해결해야 한다.

h. Clone / Pull / Push

git clone은 원격 저장소에 있는 프로젝트 전체를 내 컴퓨터로 복제해 오는 명령어이다. git push는 내가 로컬에서 작업하고 커밋한 내용을 원격 저장소에 업로드하는 것이고, git pull은 반대로 원격 저장소에 있는 최신 변경 사항을 내 로컬로 가져오는 것이다. 협업 과정에서는 보통 작업 전에 pull로 최신 상태를 받아온 뒤 작업하고, 작업이 끝나면 push로 다시 올리는 흐름을 반복하게 된다.

i. Git 설치 확인

Git을 사용하려면 먼저 컴퓨터에 Git이 설치되어 있어야 한다. 터미널에서 git --version을 입력하면 현재 설치된 Git의 버전이 출력되며, 설치가 안 되어 있다면 아무 반응이 없거나 오류가 뜬다. 리눅스에서는 apt install git, 윈도우에서는 공식 홈페이지에서 설치 파일을 받아 설치할 수 있다.

j. 사용자 정보 설정

Git으로 커밋을 남기면 그 커밋에 "누가 작성했는지"가 함께 기록되는데, 이를 위해 처음 한 번은 사용자 이름과 이메일을 등록해야 한다.

```
git config --global user.name "이름"
```

```
git config --global user.email "이메일주소"
```

--global 옵션을 붙이면 컴퓨터 전체에서 사용하는 모든 저장소에 동

일한 사용자 정보가 적용된다.

k. 저장소 생성 (git init)

작업할 폴더로 이동한 뒤 git init을 실행하면 그 폴더가 Git 저장소로 초기화된다. 이 명령어를 실행하면 폴더 안에 .git이라는 숨김 폴더가 생기는데, 여기에 앞으로의 모든 변경 기록이 저장된다.

l. 상태 확인 (git status)

git status는 현재 작업 중인 파일들의 상태를 보여주는 명령어이다. 새로 생성됐지만 아직 추적되지 않는 파일(Untracked), 수정됐지만 아직 커밋되지 않은 파일(Modified), 커밋할 준비가 된 파일(Staged) 등을 구분해서 알려준다.

m. 파일 추가 (git add)

수정한 파일을 커밋하기 전에는 먼저 git add로 스테이징(staging) 영역에 올려야 한다. git add 파일명으로 특정 파일만 올릴 수도 있고, git add .을 사용하면 변경된 파일 전체를 한 번에 올릴 수 있다. 스테이징 영역에 올린다는 것은 "이번 커밋에 이 파일을 포함시키겠다"고 지정하는 것과 같다.

n. Commit 생성 (git commit)

스테이징한 파일들을 실제 기록으로 남기는 명령어이다. git commit -m "커밋 메시지" 형태로 사용하며, 메시지에는 이번 커밋에서 무엇을 바꿨는지 간단하고 명확하게 적는 것이 좋다.

o. Commit 기록 확인 (git log)

git log는 지금까지 쌓인 커밋 기록을 시간 순서대로 보여주는 명령어이다. 각 커밋마다 고유한 해시값, 작성자, 날짜, 커밋 메시지가 함께 출력되어서 프로젝트가 어떻게 변화해왔는지 한눈에 확인할 수 있다.